

MAXIMUM CUT PROBLEM

Backtracking Algorithm – Pseudocode Report

1. Objective

Given an undirected graph, partition its vertices into two sets so that the number of edges having endpoints in different sets is maximized. A backtracking method systematically explores possible assignments of vertices to the two partitions and keeps the best cut found.

2. Input

Input is an undirected graph $G = (V, E)$, where V is the set of vertices and E is the set of edges. The graph may contain zero vertices, zero edges, disconnected components, or multiple components. For a weighted version, each edge may additionally have a non-negative weight.

3. Output

Output consists of two partitions, A and B, containing every vertex exactly once, together with the maximum cut value. The cut value is the number of crossing edges for an unweighted graph or the sum of weights of crossing edges for a weighted graph.

4. Backtracking Idea

Each vertex is assigned either to partition A or partition B. Once a vertex is assigned, edges from that vertex to already assigned vertices can be evaluated immediately. The algorithm recursively assigns the next vertex until every vertex has a partition. The best complete assignment is stored.

5. Pruning Strategy

Pruning is applied when an upper bound on the best possible completion of the current partial assignment cannot exceed the best cut already found. A simple safe upper bound is the current cut plus the number of edges that have at least one unassigned endpoint. If this bound is no better than the best known cut, the branch is abandoned. For weighted graphs, the corresponding bound uses the sum of weights of undecided edges.

6. Structured Pseudocode

```
ALGORITHM MAXIMUM-CUT-BACKTRACKING( $G = (V, E)$ )

INPUT:
   $G$  = undirected graph
   $V = \{v_1, v_2, \dots, v_n\}$ 
   $E$  = set of edges  $(u, v)$ 

OUTPUT:
  Partition A, Partition B
  Maximum cut value bestCut

VARIABLES:
  side[1..n]      // 0 = unassigned, 1 = A, 2 = B
  bestSide[1..n]
  bestCut = 0

FUNCTION EdgeCrosses( $u, v$ ):
  IF side[ $u$ ] != 0 AND side[ $v$ ] != 0 AND side[ $u$ ] != side[ $v$ ]:
    RETURN 1
  ELSE:
    RETURN 0

FUNCTION CurrentCut():
  cut = 0

  FOR each edge  $(u, v)$  IN  $E$ :
    IF EdgeCrosses( $u, v$ ) == 1:
```

```

        cut = cut + 1

RETURN cut

FUNCTION UpperBound(currentCut):
    undecidedEdges = 0

    FOR each edge (u, v) IN E:
        IF side[u] == 0 OR side[v] == 0:
            undecidedEdges = undecidedEdges + 1

    RETURN currentCut + undecidedEdges

FUNCTION BACKTRACK(index, currentCut):
    IF index > n:
        IF currentCut > bestCut:
            bestCut = currentCut

            FOR i = 1 TO n:
                bestSide[i] = side[i]

    RETURN

    bound = UpperBound(currentCut)

    IF bound <= bestCut:
        RETURN

    side[index] = 1
    added = 0

    FOR each neighbor u OF vertex[index]:
        IF side[u] == 2:
            added = added + 1

    BACKTRACK(index + 1, currentCut + added)

    side[index] = 2
    added = 0

    FOR each neighbor u OF vertex[index]:
        IF side[u] == 1:
            added = added + 1

    BACKTRACK(index + 1, currentCut + added)

    side[index] = 0

MAIN:

    IF V is empty:
        bestCut = 0
        A = empty
        B = empty
        RETURN A, B, bestCut

    FOR i = 1 TO n:
        side[i] = 0
        bestSide[i] = 0

    BACKTRACK(1, 0)

    A = empty set
    B = empty set

    FOR i = 1 TO n:
        IF bestSide[i] == 1:
            ADD vertex[i] TO A
        ELSE:
            ADD vertex[i] TO B

    RETURN A, B, bestCut

```

7. How the Algorithm Works

The recursion creates a binary decision tree. At each level, the current vertex is placed in partition A and then partition B. When a vertex is placed, only edges connecting it to previously assigned vertices can immediately increase the cut. This avoids repeatedly scanning the entire edge set during every recursive call.

8. Correctness

Every vertex has exactly two possible assignments, so the recursion represents every possible two-way partition. A complete branch therefore corresponds to one valid partition. The algorithm updates the best solution whenever a complete partition has a larger cut. The pruning bound is safe because every undecided edge can contribute at most one crossing edge. Thus a pruned branch cannot contain a solution better than the current best solution. Consequently, the returned partition is optimal.

9. Handling All Cases

The algorithm handles an empty graph by returning two empty partitions and cut value zero. A graph with vertices but no edges also has cut value zero. Disconnected graphs are handled naturally because each component can be assigned independently within the same search. A graph with all edges inside one partition produces fewer crossing edges than a partition that separates their endpoints. The same framework can be extended to weighted graphs by replacing edge counts with non-negative edge weights.

10. Computational Complexity

Without pruning, there are 2^n possible assignments for n vertices, giving exponential time in the worst case. With adjacency lists, the incremental cut update examines the already assigned neighbors of the selected vertex. The pruning rule can dramatically reduce the number of explored branches for favorable graphs, but the worst-case complexity remains exponential because Maximum Cut is an NP-hard optimization problem. Space complexity is $O(n + |E|)$ for the graph and $O(n)$ additional space for the recursion and partition arrays.

11. Improvements

The basic upper bound can be strengthened using graph-specific bounds. A good initial solution from a greedy or local-search method gives a larger bestCut early, making pruning more effective. Vertices can also be ordered by degree so that high-impact decisions are explored earlier. For large graphs, local search, simulated annealing, semidefinite-programming approximations, or other heuristics are generally more practical than exhaustive backtracking.

12. Conclusion

Backtracking provides a clear exact strategy for the Maximum Cut Problem. It explores both possible partitions for every vertex, computes incremental cut contributions, and uses an admissible upper bound to eliminate branches that cannot improve the current solution. The method is correct and handles general undirected graphs, but its exponential worst-case behavior limits its use to small or moderate instances. Stronger bounds and heuristic initial solutions can improve practical performance.

13. Algorithm Summary

Step	Action
1	Read the undirected graph.
2	Initialize all vertices as unassigned.
3	Choose the next vertex.
4	Try placing it in partition A.
5	Try placing it in partition B.
6	Update the current cut using already assigned neighbors.

7	Compute an upper bound and prune if it cannot beat bestCut.
8	When all vertices are assigned, update the best partition.
9	Return A, B and the maximum cut value.